Foundations of Machine Learning and Deep Learning

3. Linear Regression



upna
Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

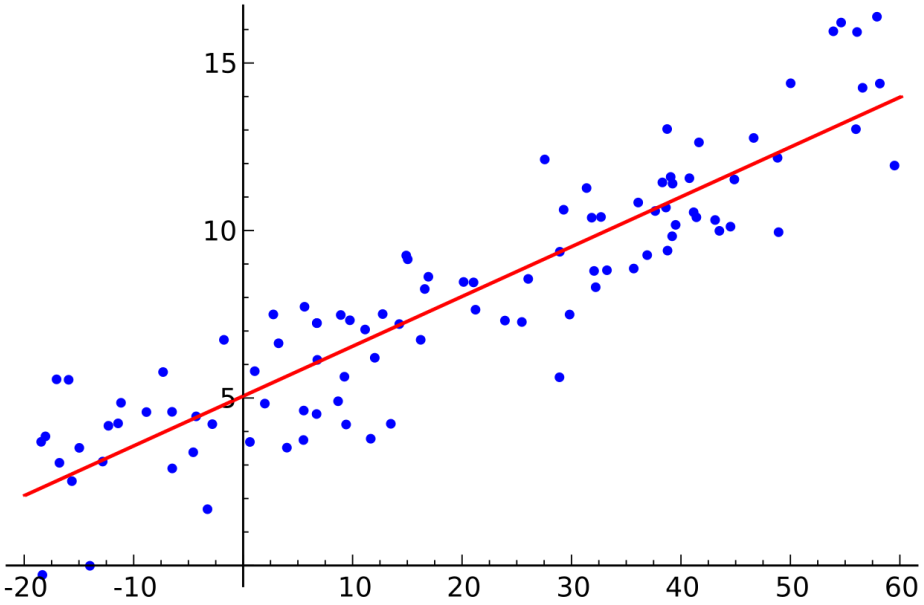
Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

Introduction

- **Linear regression** dates back to Gauss, Legendre, etc. (19th century)
- Simple yet powerful enough for real tasks

- It is a parametric model
  - Introduces a key cycle: model → loss → optimization



Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

Model

- **Main idea:**
  - We want to learn a function that maps input features **x** to an output $\hat{y}$
- **Model assumption**

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b$$

  - Each feature x_j contribues proportionally via its weight w_j
  - Bias b = baseline prediction when all features are zero.

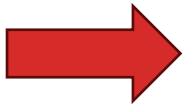
Model

Example

- Suppose we want to predict house prices (*y*) from:
  - x_1 = house size (m^2)
  - x_2 = number of rooms
- Model:

$$\hat{y} = w_1 \cdot x_1 + w_2 \cdot x_2 + b$$

- Example parameters:
  - $w_1 = 3000$ (price per m^2)
  - $w_2 = 10000$ (extra value per room)
  - $b = 50000$ (baseline value)



Prediction for a $100m^2$, 3-room house:

$$\hat{y} = 3000 \cdot 100 + 10,000 \cdot 3 + 50,000$$

$$= 380,000$$

Model

- **Model in vector form (single example)**

$$\hat{y} = \mathbf{x}^\top \mathbf{w} + b$$

where

- $\mathbf{x} = [x_1, x_2, \ldots, x_d]^\top$
- $\mathbf{w} = [w_1, w_2, \ldots, w_d]^\top$
- $b \in \mathbb{R}$

- **Model in matrix form (dataset)**

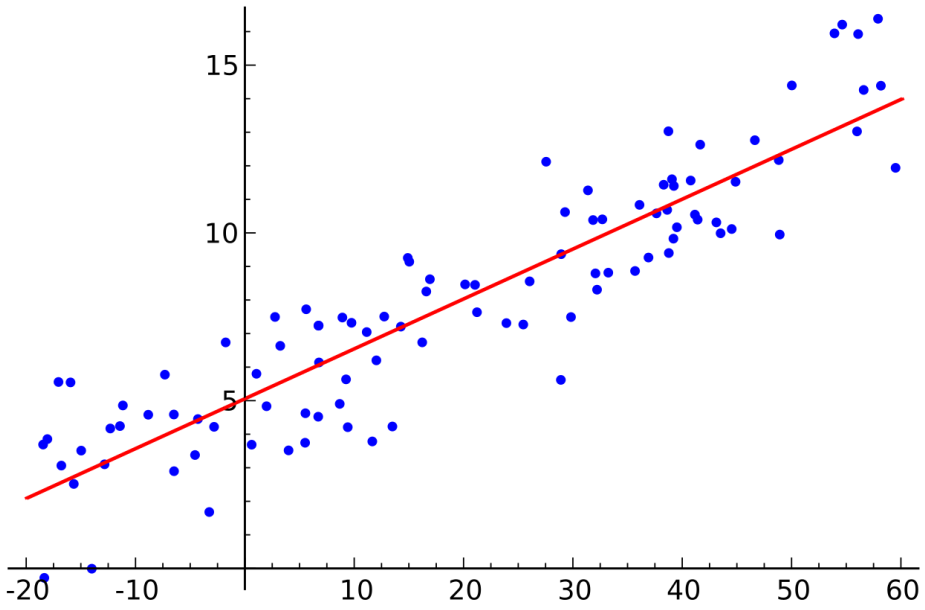
$$\hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}$$

$$X = \begin{bmatrix} - & (\mathbf{x}^{(1)})^\top & - \\ - & (\mathbf{x}^{(2)})^\top & - \\ & \vdots & \\ - & (\mathbf{x}^{(n)})^\top & - \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(n)} & x_2^{(n)} & \cdots & x_d^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times d} \quad \Rightarrow \quad \hat{\mathbf{y}} = \begin{bmatrix} (\mathbf{x}^{(1)})^\top \mathbf{w} + b \\ (\mathbf{x}^{(2)})^\top \mathbf{w} + b \\ \vdots \\ (\mathbf{x}^{(n)})^\top \mathbf{w} + b \end{bmatrix} \in \mathbb{R}^{n \times 1}$$

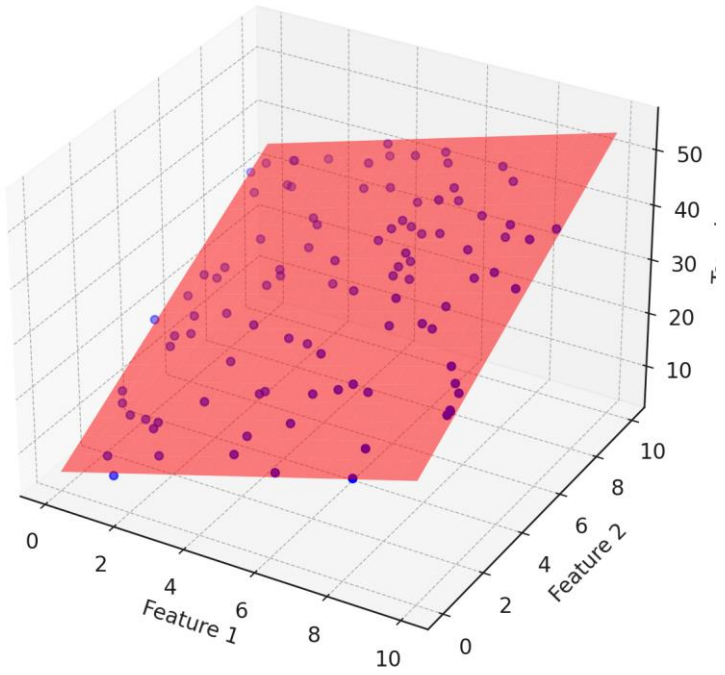
Model

- **Geometric interpretation**
  - In general: Fitting a *d*-dimensional hyperplane embedded in $\mathbb{R}^{d+1}$

For 2D: regression line in plane
$d = 1$



For 3D: regression plante through points
$d = 2$

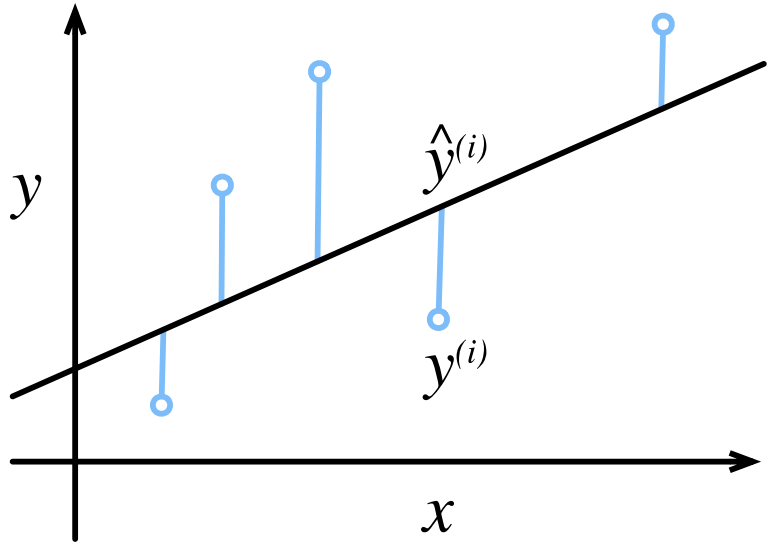


Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

Loss function

- **Residuals**
  - We need a measure of how far predictions are from true targets
  - Residuals: $r^{(i)} = y^{(i)} - \hat{y}^{(i)}$



- **Learning:** Choosing parameters (**w**, *b*) that make residuals close to 0 for all examples

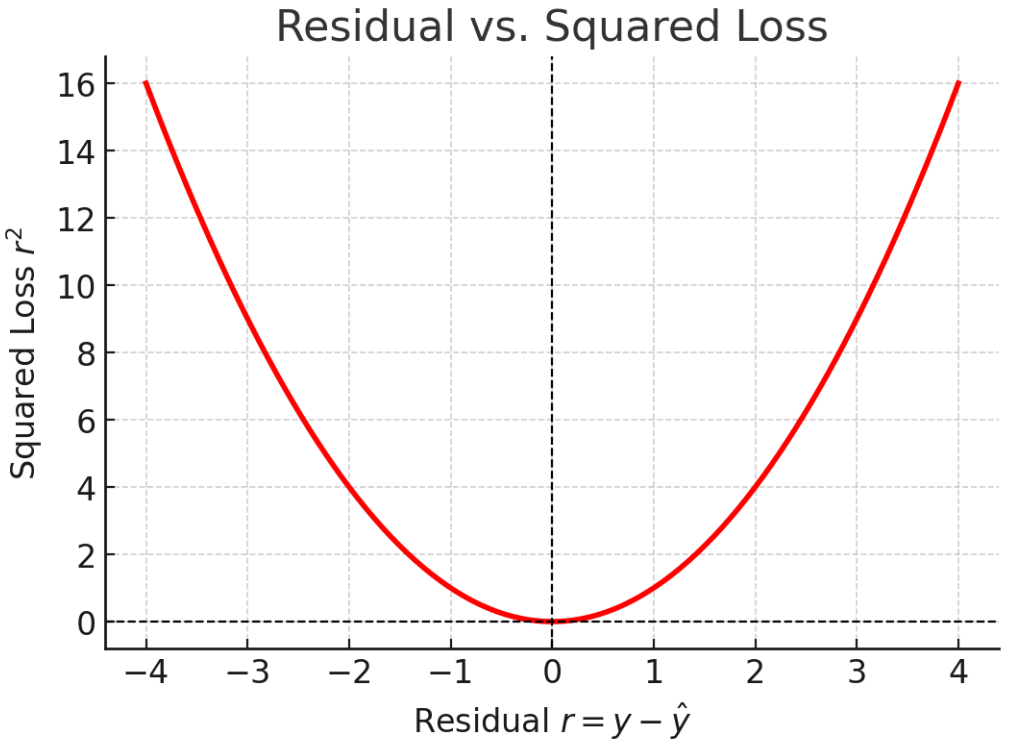
Loss function

- **Squared error**
  - For a single sample i

$$\ell^{(i)}(\mathbf{w}, b) = \left(y^{(i)} - \hat{y}^{(i)}\right)^2$$

  - Always non-negative
  - Penalizes larger errors more strongly
  - Differentiable and convex

- For a dataset: average of squared errors



Loss function

- **Mean squared error (MSE)**
  - For a dataset: average of squared errors

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

  - In matrix form:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \| \mathbf{y} - (X\mathbf{w} + b\mathbf{1}) \|_2^2$$

- **Why MSE?**
  - Statistical background: comes from Gaussian noise assumption
  - Convex and differentiable
  - Easy to compute and interpretable

Loss function

- **Other loss functions**
  - Mean Absolute Error (MAE)
    - More robust to outliers
    - Not differentiable at $r = 0$

$$\mathcal{L}_{\text{MAE}}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left| y^{(i)} - \hat{y}^{(i)} \right|$$

  - Huber Loss (parameter $\delta > 0$)
    - Quadratic near 0, linear for large residuals
    - Smooth compromise between MSE and MAE

$$\ell_\delta(r) = \begin{cases} \frac{1}{2} r^2 & \text{if } |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & \text{if } |r| > \delta \end{cases}$$

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

Probabilistic view

- **Assumption**
  - Target variable can be explained as “deterministic linear part + noise”

$$y^{(i)} = (\mathbf{x}^{(i)})^\top \mathbf{w} + b + \varepsilon^{(i)} \qquad \text{with } \varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$$

Constant variance

$$\Rightarrow \quad y^{(i)} \mid \mathbf{x}^{(i)} \sim \mathcal{N}\big((\mathbf{x}^{(i)})^\top \mathbf{w} + b, \sigma^2\big)$$

  - Thus, density for a single sample:

$$p(y^{(i)} \mid \mathbf{x}^{(i)}, \mathbf{w}, b) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(- \frac{(y^{(i)} - \hat{y}^{(i)})^2}{2\sigma^2} \right)$$

Probabilistic view

- **Likelihood of the dataset**

The likelihood measures how plausible the observed data is, for a given set of parameters

$$L(\mathbf{w}, b, \sigma^2 \mid X, \mathbf{y}) = p(\mathbf{y} \mid X, \mathbf{w}, b, \sigma^2)$$

$$p(\mathbf{y} \mid X, \mathbf{w}, b, \sigma^2) = \prod_{i=1}^{n} p(y^{(i)} \mid \mathbf{x}^{(i)}, \mathbf{w}, b, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(- \frac{(y^{(i)} - \hat{y}^{(i)})^2}{2\sigma^2} \right)$$

Maximizing the likelihood is equivalent to maximizing the log-likelihood

Probabilistic view

- **Log-Likelihood of the dataset**

$$p(\mathbf{y} \mid X, \mathbf{w}, b, \sigma^2) = \prod_{i=1}^{n} p(y^{(i)} \mid \mathbf{x}^{(i)}, \mathbf{w}, b, \sigma^2) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(- \frac{(y^{(i)} - \hat{y}^{(i)})^2}{2\sigma^2} \right)$$

$$\rightarrow \quad \ell(\mathbf{w}, b) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$$

$$\rightarrow \quad \textbf{Maximizing the likelihood is equivalent to minimizing} \quad \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$$

Probabilistic view

- **Conclusion**
  - The MSE loss is not arbitrary
    - It comes from the Gaussian noise assumption
  - Other distributions correspond to other models:
    - Bernouilli likelihood → logistic regression (next day)
    - Categorical likelihood → softmax regression (soon)
  - This establishes a probabilistic foundation for linear models

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

How are the parameters learnt? – Analytic solution

- **Objective recap**
  - We want to minimize the MSE loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \| \mathbf{y} - (X\mathbf{w} + b\mathbf{1}) \|_2^2$$

  - We can do a trick: Augmented matrices

$$\tilde{X} = \begin{bmatrix} 1 & (\mathbf{x}^{(1)})^\top \\ 1 & (\mathbf{x}^{(2)})^\top \\ \vdots & \vdots \\ 1 & (\mathbf{x}^{(n)})^\top \end{bmatrix} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(n)} & \cdots & x_d^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times (d+1)} \qquad \tilde{\mathbf{w}} = \begin{bmatrix} b \\ \mathbf{w} \end{bmatrix} = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_d \end{bmatrix} \in \mathbb{R}^{(d+1) \times 1}$$

$$\hat{\mathbf{y}} = \tilde{X}\,\tilde{\mathbf{w}}$$

How are the parameters learnt? – Analytic solution

- **Optimization problem**
  - We want

$$\tilde{\mathbf{w}}^* = \arg\min_{\tilde{\mathbf{w}}} \|\mathbf{y} - \tilde{X}\tilde{\mathbf{w}}\|_2^2$$

$$=$$

$$\mathcal{L}(\tilde{\mathbf{w}}) = (\mathbf{y} - \tilde{X}\tilde{\mathbf{w}})^\top (\mathbf{y} - \tilde{X}\tilde{\mathbf{w}})$$



derivate

$$\nabla_{\tilde{\mathbf{w}}}\mathcal{L} = -2\tilde{X}^\top(\mathbf{y} - \tilde{X}\tilde{\mathbf{w}})$$



set =0

$$\tilde{X}^\top\tilde{X}\,\tilde{\mathbf{w}} = \tilde{X}^\top\mathbf{y}$$

How are the parameters learnt? – Analytic solution

- **Closed form solution**

$$\tilde{X}^\top \tilde{X}\, \tilde{\mathbf{w}} = \tilde{X}^\top \mathbf{y}$$

  - If $\tilde{X}^\top \tilde{X}$ is invertible:

$$\boxed{\tilde{\mathbf{w}}^* = (\tilde{X}^\top \tilde{X})^{-1} \tilde{X}^\top \mathbf{y}}$$

  - This gives the optimal parameters in one step, but
    - Requires computing the inverse (computationally expensive) $(O(d^3))$
    - Fails if the matrix is singular (multicollinearity)
    - Not suitable for high dimensional data

Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

How are the parameters learnt? – Gradient descent

- **Why gradient descent?**
  - Analytic solution exists, but
    - Requires matrix inversion (expensive)
    - Can be infeasible for high dimensions
  - Other models (neural networks) don't have closed-forms solutions
  - We need some optimization algorithm to find the parameters
  - **Idea**: iteratively improve parameters using the slope of the loss function
    - How? Using the gradient!

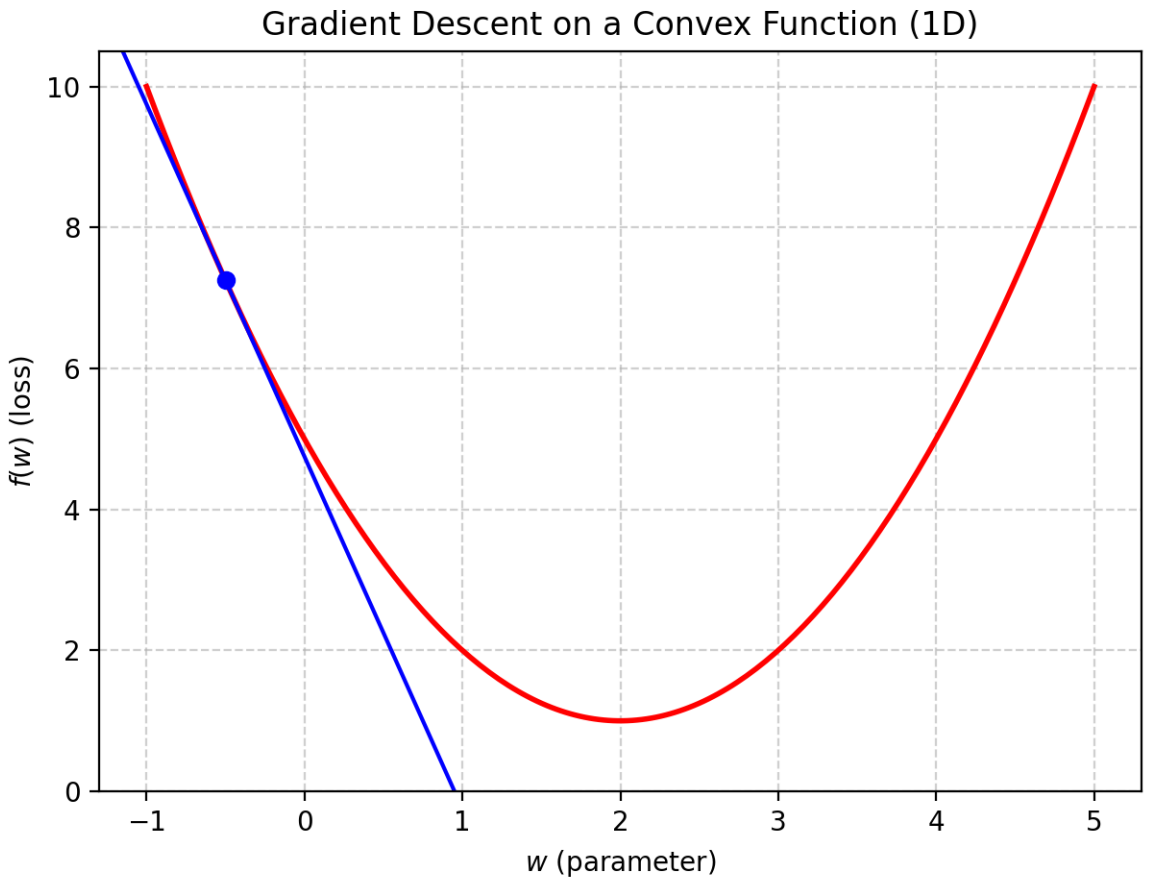
How are the parameters learnt? – Gradient descent

- **Idea**
  - Gradient = slope = tells the direction of steepest increase

$$f(w)$$

$$\nabla f(w) = \frac{d}{dw} f(w)$$

Gradient points “uphill”

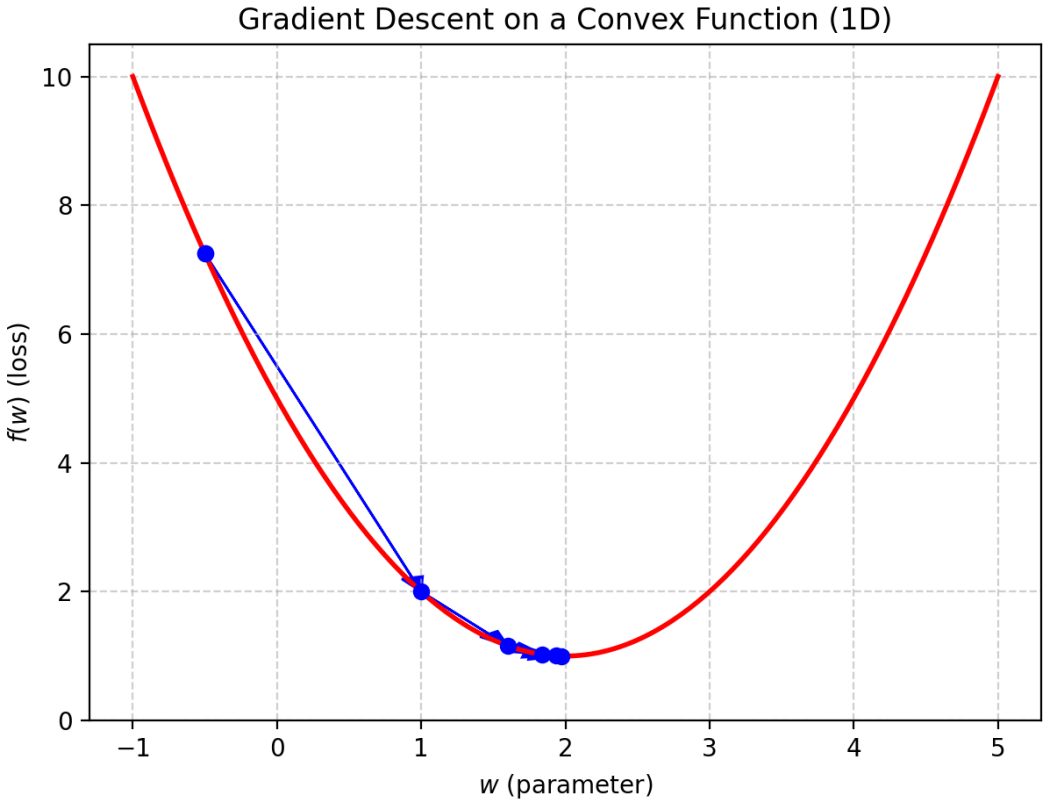


How are the parameters learnt? – Gradient descent

- **Idea**
  - Start at some initial guess $w^{(0)}$
  - **Step in the opposite direction of the gradient**

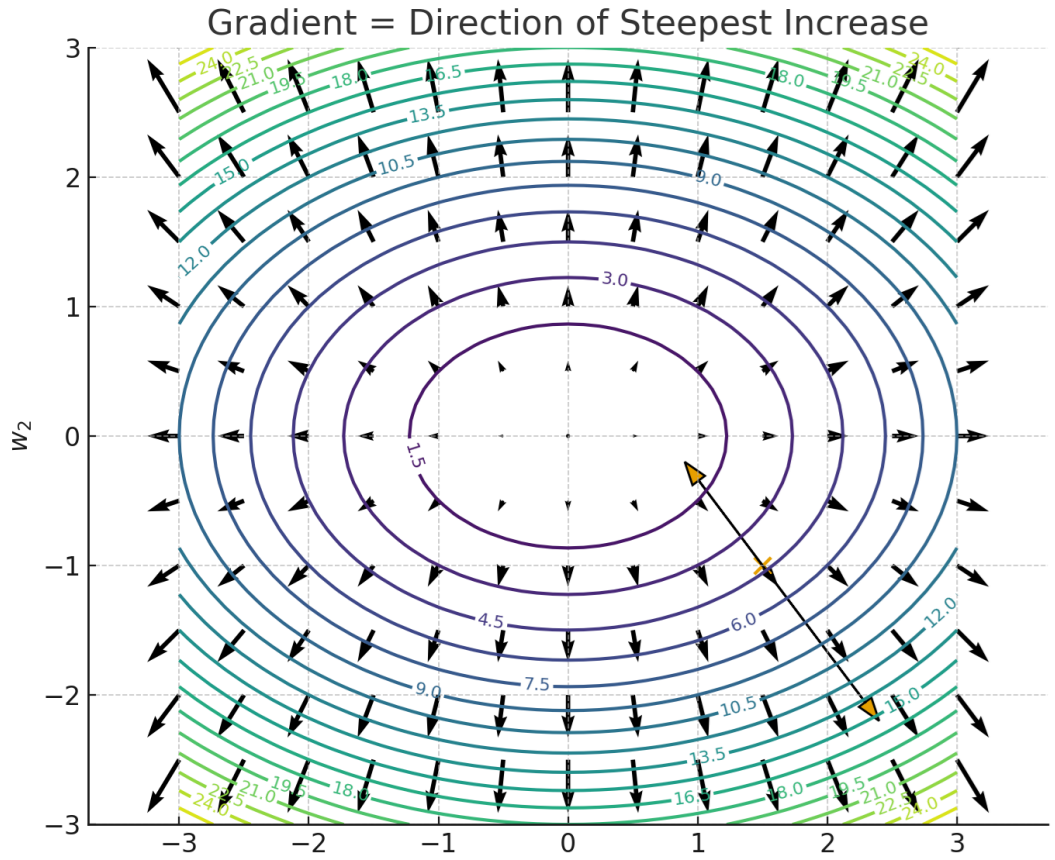
$$w \leftarrow w - \eta \nabla f(w)$$

Learning rate $\eta > 0$ controls step size.



How are the parameters learnt? – Gradient descent

- **Idea**
  - In higher dimensions the same principle holds
    - Gradient = direction of steepest increase



How are the parameters learnt? – Gradient descent

- **Gradient descent for linear regression**
  - For a dataset $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{n}$

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n}\left(y^{(i)} - \left(\sum_{j=1}^{d} w_j x_j^{(i)} + b\right)\right)^2$$

  - Gradient w.r.t. weight w_j:

$$\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right) x_j^{(i)}$$

  - Gradient w.r.t. bias b:

$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}\left(y^{(i)} - \hat{y}^{(i)}\right)$$

How are the parameters learnt? – Gradient descent

- **Gradient descent for linear regression**
  - Update each weight:

$$w_j \leftarrow w_j - \eta \cdot \frac{\partial \mathcal{L}}{\partial w_j}, \quad j = 1, \ldots, d$$

  - Update bias:

$$b \leftarrow b - \eta \cdot \frac{\partial \mathcal{L}}{\partial b}$$

  - Repeat until convergence (or max iters reached)

Later in the course we will see variants of Gradient descent (Batch GD, Stochastic GD (SGD), Mini-batch GD...)

How are the parameters learnt? – Gradient descent

- **Note: The importance of scaling**
  - The scale of the features has a big impact on the gradient
  - Gradient descent updates:

$$\Delta w_j \propto \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)}) x_j^{(i)}$$

  - Larger scale features dominate updates
    - Slower convergence (zig-zag)
    - Numerical instability

How are the parameters learnt? – Gradient descent

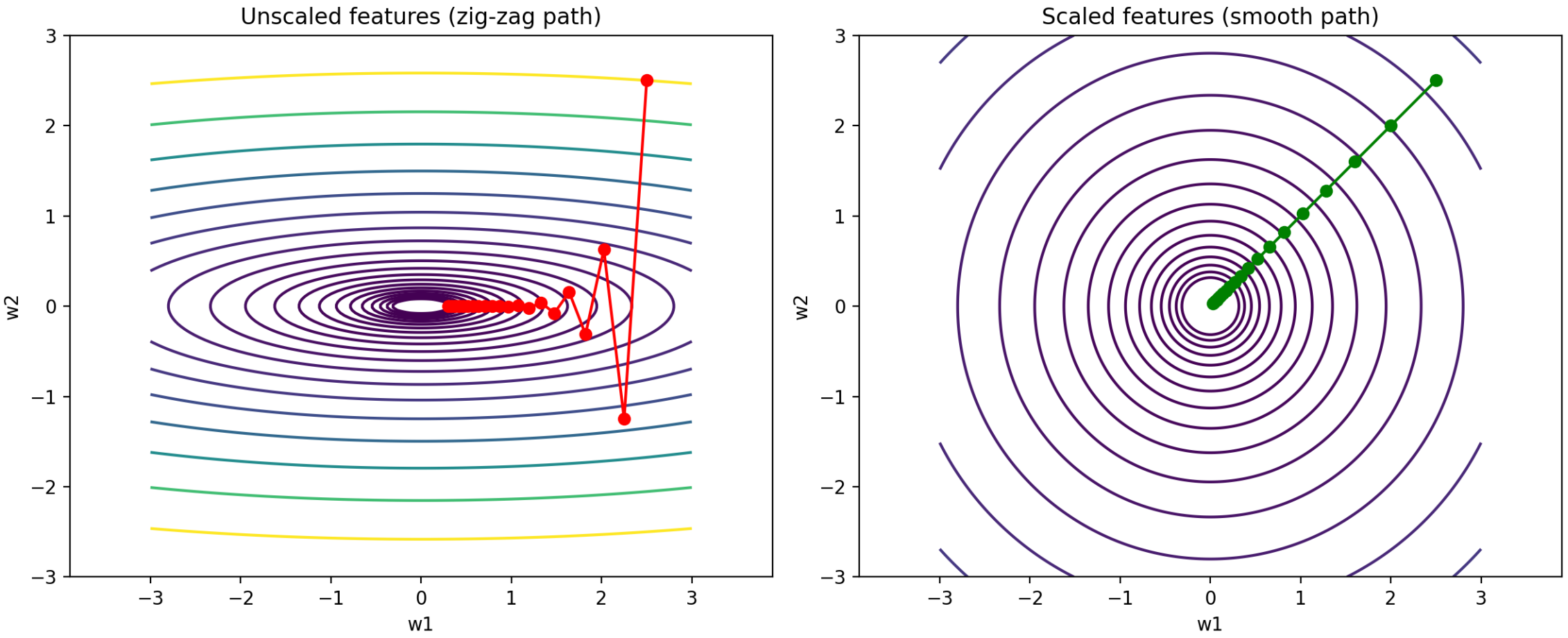
- **Solution: feature scaling**
  - Standard approach: standardization

$$x_j \longmapsto \frac{x_j - \mu_j}{\sigma_j}$$

    - Features are centered around 0, with unit variance
    - Loss contours become more “circular”
    - Gradient descent converges faster and more stably

How are the parameters learnt? – Gradient descent

- **The importance of scaling**



Outline

- Introduction
- Model
- Loss function
- Probabilistic view
- How are the parameters learnt?
  - Analytic solution
  - Gradient descent
- Implementation

Implementation – scikit-learn

- **Scikit-learn** (Usual mechanics: .fit(), .predict()…)
  - LinearRegression: Solves ordinary least squares with analytic solution
  - SGDRegressor: Fits a linear model using stochastic gradient descent
  - Ridge: Adds L^2 regularization $\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha\|\mathbf{w}\|_2^2$
  - Lasso: Adds L^1 regularization $\sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha\|\mathbf{w}\|_1$

Implementation – PyTorch

- **PyTorch**
  - We need to translate the math into code (no high-level abstractions)
  - We will implement the training loop:
    - Data setup
    - Initialize parameters
    - Loop:
      - Forward pass (make predictions)
      - Compute loss
      - Backward pass (gradients)
      - Update parameters

Implementation – PyTorch

- **Training loop**
  - Data setup

Guarantee consistency (avoid mismatch errors)

By default, PyTorch uses torch.float32, but converting from NumPy it inherits de dtype (by default float64)

```
import torch

# Training data (n samples, d features)
X_t = torch.tensor(X_train, dtype=torch.float32)   # shape (n, d)
y_t = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)  # shape (n, 1)
```

Numpy arrays with training data

Make sure it has column form for broadcasting

Implementation – PyTorch

- **Training loop**
  - Initialize parameters

```
# Parameters with requires_grad=True
w = torch.zeros((X_t.shape[1], 1), requires_grad=True)  # (d,1)
b = torch.zeros(1, requires_grad=True)                  # (1,)
```

Parameters are going to be learnt, we need to keep grads

Implementation – PyTorch

- **Training loop**
  - Loop

```
lr = 1e-2
epochs = 100
```
→ Set learning rate and number of epochs

```
for epoch in range(epochs):
```
→ Loop

```
    # 1. Forward pass
    y_hat = X_t @ w + b
```
→ Forward pass

```
    # 2. Loss (MSE)
    loss = ((y_t - y_hat) ** 2).mean()
```
→ Compute loss

```
    # 3. Backward pass
    loss.backward()
```
→ Backward pass (autograd computes gradients)

```
    # 4. Parameter update
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad
```
→ Update parameters (gradient descent)

```
    # 5. Zero gradients
    w.grad.zero_()
    b.grad.zero_()
```
→ Reset gradients for the next iteration

Not differentiable. PyTorch would try to add that operation into the graph (error)

Implementation – PyTorch

- **Key takeaways**
  - Implementation of linear regression from scratch in PyTorch
  - This training loop (forward → loss → backward → update) is the core of all neural network training
  - In future lectures, we'll use more PyTorch's tools to make training easier
    - nn.Module, optim, nn.MSELoss, DataLoader…